

Unit 11: Arrays

CONTENTS

Objectives

Introduction

11.1 Arrays

11.2 Types of Arrays

11.3 Array Declaration

11.4 Array Initialization

11.5 Accessing Elements of an Array

11.6 Passing Array as an Argument to Function

Summary

Keywords

Self Assessment

Answers for Self Assessment

Review Questions

Further Readings

Objectives

After studying this unit, you will be able to:

- Explain arrays
- Describe two dimensional array
- Describe array initialization

Introduction

An array is a group of data items of same data type that share a common name. Ordinary variables are capable of holding only one value at a time. If we want to store more than one value at a time in a single variable, we use arrays.

An array is a collective name given to a group of similar quantities. Each member in the group is referred to by its position in the group.

Arrays are allocated the memory in a strictly contiguous fashion. The simplest array is one-dimensional array which is simply a list of variables of same data type. An array of one-dimensional arrays is called a two-dimension array.

11.1 Arrays

Arrays are allocated the memory in a strictly contiguous fashion. The simplest array is one-dimensional array which is a list of variables of same data type. An array of one dimensional arrays is called a two-dimensional array; array of two-dimensional arrays is three-dimensional array and so on.

The members of the array can be accessed using positive integer values (indicating their order in the array) called subscript or index. Look at an array of integers as shown below:

200	120	-78	100	0
a[0]	a[1]	a[2]	a[3]	a[4]

The description of this array is listed below:

Name of the array : a

Data type of the array : integer

Number of elements : 5

Valid index values : 0, 1, 2, 3, 4

Value stored at the location a[0] : 200

Value stored at the location a[1] : 120

Value stored at the location a[2] : -78

Value stored at the location a[3] : 100

Value stored at the location a[4] : 0

Advantages of Arrays

Arrays offer a number of advantages, some of which are elucidated below:

1. If only a limited number of variables of a particular data type is required in a program, one can choose the variable names to suite the situation. Let us say we require five integer type variables, we can define them as follows:

```
int v_one, v_two, v_three, v_four, v_five;
```

Now, consider if we require hundred integer type variables, is the above approach convenient? Obviously not. We can, instead, use an array of integer type having 100 elements as shown below:

```
int num[100];
```

2. Array elements can be accessed using index. Therefore, all the elements can be processed in a desired manner in a single for loop that runs for each element, as shown below:

```
for(i=0; i<100; i++)
```

```
num[i]=num[i]+10;
```

In a single for loop, all the elements have been incremented by 10.

3. Since array elements are physically created contiguously in the memory, they can be accessed using pointers (as you will learn later). Therefore, there are more than one way to reference array elements.

11.2 Types of Arrays

According to the number of subscripts required to access an array element, arrays can be of following types:

1. One-dimensional array
2. Multi-dimensional array

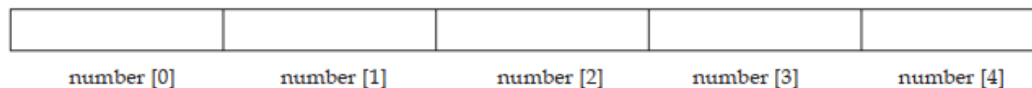
One-dimensional Array

A list of items can be given one variable name using only one subscript and such a variable is called a one dimensional array.

Example: If we want to store a set of five numbers by an array variable number. Then it will be accomplished in the following way:

```
int number [5];
```

This declaration will reserve five contiguous memory locations capable of storing an integer type value each, as shown below:



As C performs no bounds checking, care should be taken to ensure that the array indices are within the declared limits. Also, indexing in C begins from 0 and not from 1.

Two-dimensional and Multi-dimensional Array

It is possible to have an array of more than one dimensions. Two dimensional array (2-D array) is an array of number of 1-dimensional arrays.

A two dimensional array is also called a matrix. Consider the following table:

	Item1	Item2	Item3
Sales 1	300	275	365
Sales 2	210	190	325
Sales 3	405	235	240
Sales 4	260	300	380

This is a table of four rows and three columns. Such a table of items can be defined using two dimensional arrays.

General form of declaring a 2-D array is

```
data_type array_name [row_size] [column_size];
```



Example: `int marks [4] [2];`

It will declare an integer array marks of four rows and two columns. An element of this array can be accessed by the manipulation of both the indices. `printf ("%d", marks [2] [1])` will print the element present in third row and second column.

C allows arrays of three or more dimensions. Multi-dimensional arrays are defined in much the same manner as one-dimensional arrays, except that a separate pair of square brackets is required for each subscript.

The general form of a multi-dimensional array is

```
data_type array_name [s1] [s2] [s3] . . . [sm];
```

E.g.: `int survey [3] [5] [12];`

`float table [5] [4] [5] [3];`

Here, survey is a 3-dimensional array declared to contain 180 integer type elements. Similarly, table is a 4-dimensional array containing 300 elements of floating point type.

Let us consider some applications of multidimensional array programming.

1. Sorting an integer array.

```
# include <stdio.h>
```

```
void main( )
```

```
{
```

```
int arr [5];
```

```
int i, j; temp;
```

```
printf ("\n Enter the elements of the array:");
```

```

scanf ("%d", &arr [i]);
for (i = 0; i <= 4; i ++);
{
for (J = 0; J <= 3; J ++ )
if (arr [J] > arr [J+1])
{
temp = arr [J];
arg [J] = arr [J+1];
arr [J+1] = temp;
}
}
printf ("\ n The Sorted array is:");
for (i = 0; i < 5; i++)
printf ("\ t %d", arr [i]);
}

```

2. To insert an element into an existing sorted array (Insertion Sort).

```

# include <stdio.h>
main( )
{
int i, k, y, x [20], n;
for (i = 0; i < 20; i++)
x [ i] = 0;
printf ("\ Enter the number of items to be inserted:\n");
scanf ("%d", &n);
printf ("\n Input %d values \n", n);
for (k = 0; k < n; k++)
{
scanf ("%d", &x [k]);
y = x [x]
for (i = k-1; i >= 0 && y < x [i]; i --)
x [i+1] = x[i];
x [i+1] = y;
}
printf ("\n The sorted numbers are:");
for (i = 0; i < n; i++)
printf ("\n %d", x [i]);
}

```

3. Accept character string and find its length.

We will solve this question by looping instead of using Library function strlen().

```
# include <stdio.h>
```

```

void main( )
{
char name [20];
int i, len;
printf ("\n Enter the name:");
scanf ("%s", name);
for (i = 0; name [i] != '\0'; i++);
Len = i - 1;
printf ("\n Length of array is % d", len);
}

```

Character Arrays

Just as a group of integers can be stored in an integer array, group of characters can be stored in a character array or "strings". The string constant is a one dimensional array of characters terminated by null character ('\0'). This null character '\0' (ASCII value 0) is different from 'O'

(ASCII value 48).

The terminating null character is important because it is the only way the function that works with string can know where the string ends.



Example: Static char name [] = {'K', 'R', 'I', 'S', 'H', '\0'};

This example shows the declaration and initialization of a character array. The array elements of a character array are stored in contiguous locations with each element occupying one byte of memory.

K	R	I	S	H	N	A	'\0'
4001	4002	4003	4004	4005	4006	4007	4009

11.3 Array Declaration

Arrays are defined in the same manner as ordinary variables, except that each array name must be accompanied by the size specification.

The general form of array declaration is:

```
data_type array_name [size];
```

data-type specifies the type of array, size is a positive integer number or symbolic constant that indicates the maximum number of elements that can be stored in the array.



Example: float height [50];

This declaration declares an array named height containing 50 elements of type float. The compiler will interpret first element as height [0]. As in C, the array elements are indexed

for 0 to [size-1].

Two dimensional arrays can be declared similarly, as shown below:

```
data_type array_name[size1][size2];
```

For instance, the following array (named b) is array of 2 arrays of integer type of size 5 elements:

```
int b[2][5];
```

The array b has 10 (2 * 5) elements, each capable of storing an integer type data, referenced as:

```
b[0][0] b[0][1] b[0][2] b[0][3] b[0][4]
b[1][0] b[1][1] b[1][2] b[1][3] b[1][4]
```

Multidimensional arrays can be declared on the similar lines. A three dimensional array (named c) of int type has been declared below:

```
Int c[2][2][5];
```

The array c has 20 ($2 * 2 * 5$) elements, each capable of storing an integer type data, referenced as:

```
c[0][0][0] c[0][0][1] c[0][0][2] c[0][0][3] c[0][0][4]
c[0][1][0] c[0][1][1] c[0][1][2] c[0][1][3] c[0][1][4]
c[1][0][0] c[1][0][1] c[1][0][2] c[1][0][3] c[1][0][4]
c[1][1][0] c[1][1][1] c[1][1][2] c[1][1][3] c[1][1][4]
```

11.4 Array Initialization

One-dimensional Array

The elements of an array can be initialized in the same way as the ordinary variables, when they are declared. Given below are some examples which show how the arrays are initialized.

```
static int num [6] = {2, 4, 5, 45, 12};
static int n [ ] = {2, 4, 5, 45, 12};
static float press [ ] = {12.5, 32.4, -23.7, -11.3};
```

In these examples note the following points:

1. Till the array elements are not given any specific values, they contain garbage value.
2. If the array is initialized where it is declared, its storage class must be either static or extern. If the storage class is static, all the elements are initialized by 0.
3. If the array is initialized where it is declared, mentioning the dimension of the array is optional.

Two-dimensional Arrays

Two dimensional arrays may be initialized by a list of initial values enclosed in braces following their declaration.

E.g.: `static int table[2][3] = {0, 0, 0, 1, 1, 1};`

initializes the elements of the first row to 0 and the second row to one. The initialization is done by row.

The aforesaid statement can be equivalently written as

```
static int table[2][3] = {{0, 0, 0}, {1, 1, 1}};
```

by surrounding the elements of each row by braces.

We can also initialize a two dimensional array in the form of a matrix as shown below:

```
static int table[2][3] = {{0, 0, 0},
{1, 1, 1}};
```

The syntax of the above statement. Commas are required after each brace that closes off a row, except in the case of the last row.

If the values are missing in an initializer, they are automatically set to 0. For instance, the statement

```
static int table [2] [3] = {{1, 1},
{2}};
```

will initialize the first two elements of the first row to one, the first element of the second row to two, and all the other elements to 0.

When all the elements are to be initialized to 0, the following short cut method may be used.

```
static int m [3] [5] = {{0}, {0}, {0}};
```

The first element of each row is explicitly initialized to 0 while other elements are automatically initialized to 0.

While initializing an array, it is necessary to mention the second (column) dimension, whereas the first dimension (row) is optional. Thus, the following declarations are acceptable.

```
static int arr [2] [3] = {12, 34, 23, 45, 56, 45};
```

```
static int arr [ ] [3] = {12, 34, 23, 45, 56, 45 };
```

Multi-dimensional Array



Example: Example of initializing a 4-dimensional array:

```
static int arr [3] [4] [2] = {{{2, 4}, {7, 8}, {3, 4}, {5, 6}},
```

```
{{7, 6}, {3, 4}, {5, 3}, {2, 3}},
```

```
{{8, 9}, {7, 2}, {3, 4}, {6, 1}, } };
```

In this example, the outer array has three elements, each of which is a two dimensional array of four rows, each of which is a one dimensional array of two elements.

11.5 Accessing Elements of an Array

Once an array is declared, individual elements of the array are referred using subscript or index number. This number specifies the element's position in the array. All the elements of the array are numbered starting from 0. Thus number [5] is actually the sixth element of an array.

Consider the program given above. It has entered 6 values in the array num. Now to read values from this array, we will again use for Loop to access each cell. The given program segment explains the retrieval of the values from the array.

```
for (count = 0; count < 6; count ++)
```

```
{
printf ("\n %d value =", num [count]);
}
```

Data can be inserted into array by treating the array elements just like any other variable. If an integer value is to be read from keyboard into an array element (say c[2][3][0]), the following code snippet would do the job:

```
Scanf("%d", &c[2][3][0]);
```

In order to read values in the entire array for loop may be used as explained by the following examples:

```
main()
```

```
{
int num [6];
int count;
for (count = 0; count < 6; count ++)
```

```
{
printf ("\n Enter %d element:" count+1);
```

```
scanf ("%d", &num [count]);
}
}
```

In this example, using the for loop, the process of asking and receiving the marks is accomplished. When count has the value zero, the scanf() statement will cause the value to be stored at num [0].

This process continues until count has the value greater than 5.



Case Study: Each element of the array has a memory address. The following program prints an array limit value and an array element address.

Program:

```
#include <stdio.h>
void printarr(int a[]);
main()
{
int a[5];
for(int i = 0;i<5;i++)
{
a[i]=i;
}
printarr(a);
}
void printarr(int a[])
{
for(int i = 0;i<5;i++)
{
printf("value in array %d\n",a[i]);
}
}
void printdetail(int a[])
{
for(int i = 0;i<5;i++)
{
printf("value in array %d and address is %16lu\n",a[i],&a[i]);
\\ A
}
}
}
```

Explanation

1. The function printarr prints the value of each element in arr.
2. The function printdetail prints the value and address of each element as given in statement A. Since each element is of the integer type, the difference between addresses is 2.
3. Each array element occupies consecutive memory locations.

4. You can print addresses using place holders %16lu or %p.

Questions

1. Write a program to add two 6 x 6 matrices.
2. Write a program to multiply any two 3 x 3 matrices.
3. Write a program to sort all the elements of a 4 x 4 matrix.
4. Write a program to obtain the determinant value of a 5 x 5 matrix.

11.6 Passing Array as an Argument to Function

If you want to pass a single-dimension array as an argument in a function, you would have to declare a formal parameter in one of following three ways and all three declaration methods produce similar results because each tells the compiler that an integer pointer is going to be received. Similarly, you can pass multi-dimensional arrays as formal parameters.

Method -1

Formal parameters as a pointer –

```
void myFunction(int *param) {
    .
    .
    .
}
```

Method -2

Formal parameters as a sized array –

```
void myFunction(int param[10]) {
    .
    .
    .
}
```

Method -3

Formal parameters as an unsized array –

```
void myFunction(int param[]) {
    .
    .
    .
}
```



Example:

Now, consider the following function, which takes an array as an argument along with another argument and based on the passed arguments, it returns the average of the numbers passed through the array as follows –

```
double getAverage(int arr[], int size)
{
    int i;
    double avg;
    double sum = 0;

    for (i = 0; i < size; ++i) {
        sum += arr[i];
    }
    avg = sum / size;
    return avg;
}
```

Now, let us call the above function as follows –

```
#include <stdio.h>

double getAverage(int arr[], int size);

int main () {
    int balance[5] = {1000, 2, 3, 17, 50};
    double avg;
    avg = getAverage( balance, 5 );
    printf( "Average value is: %f ", avg );
    return 0;
}
```

Summary

- An array is a group of memory locations related by the fact that they all have the same name and same data type.
- An array including more than one dimension is called a multidimensional array.
- The size of an array should be a positive number. If an array is declared without a size and is initialized to a series of values it is implicitly given the size of number of initializers.
- Array subscript always starts with 0. Last element's subscript is always one less than the size of the array e.g., an array with 10 elements contains element 0 to 9. Size of an array must be a constant number.

Keywords

Array: A user defined simple data structure which represents a group of same type of variables having same name each being referred to by an integral index

Multidimensional array: An array in which elements are accessed using multiple indices

One dimensional array: An array in which elements are accessed using a single index

Subscript/Index: The integral index by which an array element is accessed

Two dimensional array: An array in which elements are accessed using two indices

Self Assessment

1. Array is a group of data items of
 - A. Same data type that share a common name
 - B. Same data type that share a uncommon name
 - C. Not data type that never common name
 - D. None of the above

2. The general form of array declaration is
 - A. array_name [size];
 - B. data_type array_name [size];
 - C. data_type [size];
 - D. None

3. What will be the output of the following program if the input is - "tomorrow never comes!".

```
main()  
{  
char letter [80];  
int count;  
for (count = 0; count < 80; count++)  
letter[count] = getchar( );  
for (count = 0; count < 80; count++)  
putchar (toupper (letter[count]));  
}
```

 - A.
 - B.
 - C.
 - D.

4. The members of the array can be accessed using positive integer values called
5. While initializing a two-dimensional array, it is necessary to mention the dimension, whereas the is optional.
6. The is a one-dimensional array of characters terminated by null character ('\0').

7. All the members of an array share a common name and memory location.
 - A. True
 - B. False

8. Array elements contain garbage values till the time they are initialized.
 - A. True
 - B. False

9. 3-dimensional array declared to contain 180 integer type elements.

A. True

B. False

10. Array element can be accessed using index.

A. True

B. False

Answers for Self Assessment

- | | | | | |
|-------------------------|------|-------------------------------|--------------|--|
| 1. A | 2. B | 3. tomorrow
never
comes | 4. subscript | 5. second(column),first
dimesion(row) |
| 6. Straight
Constant | 7. B | 8. A | 9. A | 10. A |

Review Questions

1. Explain the usefulness of Arrays in C.
2. What do you mean by 'Array'? How it can be declared & initialized in a C program?
3. Draw a diagram to represent the internal storage of an Array.
4. Describe the different types of Array. Give suitable programs.
5. Find the smallest number in an array using pointers.
6. 6. If an array arr contains n elements, then write a program to check if $arr[0] = arr[n-1]$, $arr[1] = arr[n-2]$ and so on.
7. Write a program to copy the contents of one array into another in the reverse order.
8. How will you initialize a three-dimensional array $threed[3][2][3]$? How will you refer the
9. first and last element in this array?
10. Write a program to pick up the largest number from any 5 row by 5 column matrix.
11. Write a program to obtain transpose of a 4 x 4 matrix. The transpose of a matrix is obtained
12. by exchanging the elements of each row with the elements of the corresponding column.
13. Write a program that interchanges the odd and even components of an array.



Further Readings

Books Ashok N. Kamthane, "Programming with ANCI & Turbo C", Pearson Education, Year of Publication, 2008

B.W. Kernighan and D.M. Ritchie, "The Programming Language", Prentice Hall of India, New Delhi

Byron Gottfried, "Programming With C", Tata McGraw Hill Publishing Company Limited, New Delhi

Greg W Scragg, Genesco Suny, Problem Solving with Computers, Jones and Bartlett, 1997.

R.G. Dromey, Englewood Cliffs, N.J., How to Solve it by Computer, Prentice-Hall International, 1982.

Yashvant Kanetkar, Let us C



Web Links

www.en.wikipedia.org

www.web-source.net

www.webopedia.com